



AGNTCY: The Internet of Agents

AGNTCY (pronounced “agency”) is an open-source initiative launched in 2025 to build an **“Internet of Agents”** – a global, interoperable infrastructure where AI agents can discover, communicate, and collaborate across organizations ¹ ². It originated from Cisco Outshift’s vision that AI agents are being built in isolated silos and lack shared infrastructure ¹. By July 2025, over 75 companies (Cisco, Dell, Google Cloud, Oracle, Red Hat, etc.) had joined the effort and AGNTCY was donated to the Linux Foundation, making it a community-owned project ³.

AGNTCY’s **vision and mission** center on accelerating human work by enabling **agentic workflows** (multi-agent applications combining internal and third-party agents) ⁴ ². Its core goals are to establish an **open, interoperable** stack so that agents from different vendors can **discover one another and work together securely at scale** ⁵ ². To achieve this, AGNTCY defines standard data models (schemas) and protocols that cover the full multi-agent lifecycle: *discover*, *compose*, *deploy*, and *evaluate* agentic systems ⁶ ². In other words, AGNTCY aims to do for AI agents what HTTP, DNS, and other foundational protocols did for the internet ² ⁷.

Technical Objectives and Benefits

AGNTCY is explicitly designed for **interoperability, security, scalability, and standardization** across agent frameworks ⁵. Its technical objectives include: - **Interoperability:** Use common protocols so that agents from any vendor or platform can communicate seamlessly ⁵. - **Security:** Build in authentication, authorization, and encryption so multi-agent interactions are safe by default ⁵ ⁸. - **Scalability:** Leverage cloud-native architectures (e.g. OCI registries, distributed networks) so thousands of agents and messages can be handled efficiently ⁹ ¹⁰. - **Standardization:** Provide shared data models and APIs (schemas, OpenAPI specs) so agents represent capabilities in a consistent way ⁵ ¹¹.

These design goals yield clear developer benefits: agents become *easy to find and plug in*, integration complexity is reduced, and solutions can mix-and-match components from different sources. For example, a developer can publish an agent’s capabilities in AGNTCY’s directory, and other agents (even from other teams or companies) can discover and invoke it securely ⁶ ². The expected benefits include **enhanced collaboration** (agents working together in complex workflows), **improved efficiency** (reuse of specialized agents), **stronger security** (mutual authentication, data integrity), and a **future-proof architecture** that can evolve with AI innovation ⁵ ¹⁰.

Core Components and Architecture

AGNTCY is **not a single protocol** but a collection of interconnected specifications, services, and reference implementations covering the multi-agent ecosystem ¹² ². Its initial **core components** include:

- **Open Agent Schema Framework (OASF):** A **schema and metadata format** for describing agents, their capabilities, I/O, performance, identities, etc ¹³ ¹¹. OASF records (in YAML/JSON) are

essentially an agent “manifest” or description file that says *what the agent can do, what inputs it expects, and what outputs it produces* ¹¹ . Using OASF makes agents self-describing and easily **discoverable**. For example, an agent’s OASF might list skills like “document_summarization” or “image_recognition” (from a shared skill taxonomy) ¹⁴ ¹⁵ . The OASF model cleanly separates *capabilities* (what the agent does) from *constraints* (e.g. required data, cost, latency) ¹⁵ . OASF is **OCI-based and extensible**: schemas are versioned and hosted via OCI registries, and new fields can be added by community consensus ¹³ ¹⁵ . OASF can also describe non-AGNTCY protocols: for example, it supports A2A agent cards and MCP server descriptors ¹⁶ .

- **Agent Directory Service (ADS):** A **distributed directory (registry)** where agents publish their OASF metadata. The ADS lets anyone **announce** an agent’s capabilities and **discover** agents by those capabilities. For instance, an agent running on your laptop can push its OASF record (packaged as an OCI artifact) to your local ADS instance, where it gets a unique content ID (CID) based on a cryptographic hash ¹⁷ . Other agents can then query the ADS (“find me agents with skill X”) and retrieve matching records. Internally, ADS uses a *content-routing* protocol built on a Kademlia-based DHT (similar to IPFS) ¹⁸ ¹⁹ . In practice this means:
- **Skill-based discovery:** A hierarchical skill taxonomy (e.g. “vision/image_recognition”) maps queries to record IDs. Agents publish which skills they have, and ADS routes skill queries to the relevant records ¹⁴ ¹⁹ .
- **Content-addressed storage:** Each agent record is an OCI artifact named by its SHA-256 digest (a CID). This ensures **immutability** and integrity: any change to the content changes the ID ¹⁷ ²⁰ . Human-readable names (e.g. `example.com/agents/myagent`) can also be used; ADS supports verifiable domain-based naming atop CIDs ²¹ .
- **Distributed architecture:** ADS instances can federate across organizations. Each node contributes part of the DHT index and caches artifacts ²² ²³ . Queries are routed globally so you can find agents outside your own network, yet caching and replication make lookups efficient and robust ²⁴ .
- **Search features:** ADS supports flexible queries (by name, version, skill, domain, tags, etc.) and boolean filtering ²⁵ ²⁶ . Wildcards and pagination are available. Behind the scenes, ADS uses a local SQLite index for fast searching by any OASF field ²⁷ .
- **Runtime discovery:** ADS can automatically detect agents running in container environments (Kubernetes, Docker) by watching labeled pods/containers. It reads their OASF or A2A identity info and advertises them via gRPC APIs, so even ephemeral agents are discoverable ²⁸ .

Overall, ADS is like a **DNS or App Store for AI agents**. It lets one “find all agents with skill Y” across the internet of agents. By anchoring names to cryptographic CIDs and signing records, ADS satisfies the goals of being decentralized, secure, and (to an extent) human-readable – effectively addressing Zooko’s Triangle ²⁹ .

- **Agent Connect Protocol (ACP):** A **standard API spec** for invoking and interacting with agents over the network ³⁰ ¹¹ . ACP defines REST-like or RPC-like endpoints for common agent operations: load a task, invoke synchronously or asynchronously, stream outputs, supply interrupt inputs, query capabilities/schema, manage conversation threads, etc ³⁰ . It is published in OpenAPI format and includes security schemes for authentication/authorization. In essence, ACP makes every agent expose a uniform API so that any client (human or agent) can call it the same way. For example, instead of one agent having ad-hoc code to invoke another, ACP says “POST `/runs` with JSON input to start an agent run, then GET `/runs/{id}` to poll results or receive SSEs”. ACP is analogous to

LangChain's Agent Protocol or A2A, but scoped to agent invocation. Its goal is that when an agent is "plugged into" a workflow, the mechanical details of how to send tasks, handle streams, or report errors are **consistent across agents** ¹¹ ³⁰ .

- **Agent Gateway Protocol (AGP) and SLIM:** AGNTCY includes a **messaging layer** for low-latency, secure communication between agents. Two related components address this:
 - **SLIM (Secure Low-Latency Interactive Messaging):** A network protocol (and SDK) that provides encrypted, pub/sub communication for groups of agents ³¹ ³² . SLIM uses MLS (Messaging Layer Security) and supports quantum-safe crypto. It extends gRPC to allow not just request/reply but also fire-and-forget, streaming, and publish/subscribe between agents ³¹ . SLIM is intended for situations where agents need real-time interaction (e.g. human-in-the-loop updates, webhooks, or broadcast events).
 - **AGP (Agent Gateway Protocol):** A gRPC-based **network transport layer** for agent messages ³² . AGP supports multiple patterns: unary calls, client & server streaming, pub/sub, etc. It is built on HTTP/2 and Protobufs for efficiency. The AGP is used by an **Agent Gateway** component, which acts like a secure router/proxy. The Agent Gateway can mediate messages between agents, enforce policies, route pub/sub topics, and apply end-to-end encryption (including MLS/quantum-safe) ³² ³³ .

In short, SLIM and AGP provide the **communication substrate** for agent networks. They ensure that whether agents call each other directly (via ACP) or participate in event streams, all messages flow securely and efficiently.

- **Identity and Trust:** AGNTCY includes a **decentralized identity system** for agents and services. Each agent or tool can obtain a verifiable cryptographic identity (a badge or credential) from an Identity Service. This uses modern decentralized identity technologies (e.g. DIDs, VCs, or SPIFFE/SPIRE) so that organizations can issue identities that AGNTCY components can verify ³⁴ ³⁵ . In practice, this means:
 - Agents carry **identity badges** that bind their metadata (OASF records, agent cards) to a verifiable issuer. This lets other agents and registries trust that the agent is who it claims to be.
 - The **Agent Directory** uses SPIFFE/SPIRE for workload identity ³⁵ . Every ADS server and service gets a SPIFFE ID (SVID) and uses mTLS for all inter-node communication ³⁶ ³⁷ . This creates a zero-trust environment where ADS nodes must authenticate each other before exchanging data.
 - **Authentication in protocols:** ACP and AGP require clients to authenticate when calling an agent. Agents advertise what schemes they support (OAuth tokens, API keys, etc.), and clients must supply proper credentials.
 - **Data authenticity:** Every record in the ADS is cryptographically signed (e.g. via Sigstore provenance) and its hash is immutable ³⁸ . An agent's published record cannot be altered without invalidating its digest signature.

These measures ensure end-to-end trust: records are tamper-evident, agent identities are verifiable, and all network messages are encrypted and authenticated.

- **Observability and Evaluation:** AGNTCY provides tools for monitoring and assessing multi-agent applications ³⁹ . This includes telemetry collectors and evaluation frameworks that track agent performance and interactions. For instance, one can instrument an agent workflow to log execution

details, compute metrics (like completion time, success rates), and analyze them to improve efficiency over time ³⁹ .

- **Reference Implementation:** *CoffeeAGNTCY* is an example system that demonstrates how all these pieces can work together. It shows a complete stack (ADS, SLIM, ACP, identity, etc.) for building a multi-agent application end-to-end ⁴⁰ .

How Discovery and Collaboration Work

In practice, an agent developer would use AGNTCY like this:

1. **Registering an Agent:** When you build an agent (e.g. a document analysis agent), you create an OASF manifest describing it (name, version, capabilities/skills, input/output formats, etc.) ¹⁵ . You also provide any security details (e.g. authentication schemes). You package this manifest (and any code/tool images) as an OCI artifact and push it to your ADS instance. ADS computes its CID and optionally signs it. Now your agent is “announced” on the Internet of Agents ¹⁷ .
2. **Discovering Agents:** Another agent (or developer) needing a particular function can query ADS. For example, calling `directl search --skill "natural_language_processing"` will return all known agents with NLP skills. The query travels through the DHT to find matching records (a sparse semantic index), then detailed OASF records are pulled from a registry for final filtering ¹⁹ . This yields the selected agents’ network endpoints and capability info.
3. **Validation and Trust:** Before invoking any discovered agent, your agent can verify the record’s integrity and identity. The ADS record includes the agent’s public keys or identity badge, and the signature ensures the content is authentic ³⁸ . Your agent checks that the required skills match and that any trust policies (cert issuer, reputation, etc.) are satisfied.
4. **Communication:** Once an agent is chosen, your agent invokes it via the **Agent Connect Protocol (ACP)** or **Agent Gateway Protocol (AGP)**. For a simple task, it might do an HTTP POST to the agent’s `/runs` endpoint (ACP) with the task payload. The remote agent processes the task (possibly calling other internal tools or models) and streams back results. If the interaction is long-running or streaming, SLIM/AGP can provide real-time updates via SSE or gRPC streams ³⁰ ³¹ .
5. **Workflows:** AGNTCY supports chaining multiple agents into a workflow. For example, a *planner agent* could receive a user request, then use ACP to call a *search agent*, then an *analysis agent*, then a *summarizer agent*, each connected via standardized protocols. Data passing between agents may use **I/O Mapper Agents** (special AGNTCY components that semantically transform outputs to match inputs) ⁴¹ . Agents can also use **API Bridge Agents** to turn natural language instructions into concrete API calls for external services. This modular approach means agents focus on their core skills and delegate other tasks to the network of agents.

Throughout this process, AGNTCY’s standard protocols ensure that each step is predictable and secure. Errors or edge cases are handled uniformly (e.g. an ACP timeout or “capability not found” is a defined response). Because all agents speak the same “language” (data schemas, endpoints, security headers), the system is resilient and maintainable ¹¹ ⁴² .

Security and Trust Model

Security is a fundamental pillar of AGNTCY. The platform enforces **zero-trust principles** and **data authenticity** at every layer. Key points include:

- **Cryptographic Content-Addressing:** Every agent record (manifest, data blob) has a SHA-256 digest. Since the digest is a function of the content, it is self-authenticating: any tampering changes the ID ³⁸. ADS records include Sigstore signatures and provenance, so a receiver can verify *who* published the record and that it has not been altered. This prevents agents from impersonating others or swapping out payloads undetected ³⁸.
- **Mutual TLS and SPIFFE Identities:** Directory nodes and gateways use mutual TLS with SPIFFE IDs (SVIDs). SPIRE issues short-lived identities to each service, so communications between ADS nodes (or between agent gateways) are always authenticated and encrypted ³⁵ ³⁷. Operators can enforce allowlists of trusted peers or full attestation for federation. This locks down the network layer, mitigating DDoS or unauthorized peers.
- **Protocol Authentication:** When one agent calls another over ACP or AGP, it must authenticate according to the target's advertised scheme (API key, OAuth token, etc.). ACP includes security schemes in its OpenAPI spec. Unauthenticated requests are rejected with clear 401/403 responses. Long-running tasks can transition to an "authentication required" state if the callee agent needs additional credentials, ensuring policies are respected ⁴³ ⁴⁴.
- **Identity Verification:** AGNTCY envisions a system of **verifiable credentials** (agent badges) for each agent. For example, a certified agent might hold a credential from its issuing organization attesting to certain attributes. Other agents or directories can check these credentials before trusting an agent's claims. This layered identity (credentials + signatures + TLS) gives multiple ways to enforce trust and accountability.

In short, AGNTCY is built with **enterprise-grade security** in mind. Its security model is documented in depth in the ADS Trust Model and leverages modern standards (SPIFFE, mTLS, Sigstore, OAuth, etc.) ³⁵ ³⁸. The aim is that even in a decentralized, global agent network, one can enforce strict access control and non-repudiation.

Related Standards: A2A, MCP, LangChain Agent Protocol

AGNTCY does not operate in isolation. It is complementary to other agent and automation protocols:

- **Agent-to-Agent (A2A):** Google's open **A2A** protocol (released 2025) standardizes direct JSON-RPC messaging between agents over HTTP(S) ⁴⁵ ⁴⁶. A2A focuses on peer-to-peer task coordination (e.g. agent B invoking agent C via RPC). AGNTCY can integrate with A2A by including A2A AgentCards (metadata) in the directory ¹⁶ and by using common transport (HTTP/SSE). In essence, A2A could be used for low-level agent dialogue, while AGNTCY provides the discovery and data schema layer around it ⁴⁷ ⁴⁵.

- **Model Context Protocol (MCP):** Anthropic's **MCP** (late 2024) standardizes how AI models fetch context from external data sources or tools ⁴⁸ ⁴⁹. MCP is like a “USB-C for AI tools” that lets any agent plugin any database, SaaS API, or file system via a common RPC interface. AGNTCY can incorporate MCP by, for example, listing MCP server descriptions in the Agent Directory ¹⁶. An AGNTCY agent workflow could use MCP internally to access data, while still using AGNTCY protocols for inter-agent communication. In short, MCP handles *agent-to-tool* integration, while AGNTCY handles *agent-to-agent* orchestration and discovery ⁴⁷ ⁴⁸.
- **LangChain Agent Protocol:** LangChain's **Agent Protocol** (2024) is another open API spec for agent interactions. It defines REST endpoints (like `/runs`, `/threads`) very similar to ACP. AGNTCY's ACP and LangChain's protocol share a common approach. It's expected that these will remain compatible or even converge, since many contributors overlap. AGNTCY's spec is fully versioned and extensible so it can absorb LangChain's API style.

In summary, **AGNTCY is designed to play well with the broader ecosystem**. A2A might be used for agile messaging, MCP for tool hooks, LangChain protocol for certain agent orchestration, but AGNTCY glues them together by providing a unified discovery, identity, and schema framework ⁴² ⁴⁷.

Benefits and Outlook

By adopting AGNTCY standards, developers gain:

- **Easy Discovery:** Agents no longer have to hard-code each other's addresses. A developer can simply query the AGNTCY directory by capability and get back the best agents to solve a task ¹⁴ ⁶.
- **Composable Workflows:** Agents can freely call one another via standard APIs (ACP/AGP), so complex tasks can be split into specialized agents (e.g. “`finance_agent`”, “`compliance_agent`”) and composed without custom integration code ¹¹ ⁴¹.
- **Security by Design:** Mutual authentication (SPIFFE TLS) and signed records mean agents only talk to known, verified peers. Data shared between agents can be traced to its source via signatures ³⁸ ³⁵.
- **Interoperability Across Platforms:** An AGNTCY-compatible workflow can mix agents written in Python, Java, or any language, hosted on-prem or in the cloud, and they'll still fit into the same directory and messaging fabric ⁴² ⁵⁰.
- **Ecosystem Growth:** Since AGNTCY is open-source and governed by a Linux Foundation community, tools and services (like registries, SDKs, UIs) will evolve organically. Early members (Cisco, LangChain, Galileo, Oracle, etc.) will bring their technologies (SLIM, LangGraph orchestration, cloud SDKs, etc.) into the fold.

As a whole, AGNTCY represents an **ambitious, full-stack approach** to unifying AI agents. It goes beyond point solutions by aiming to standardize discovery, identity, communication, and security – essentially laying down the Internet's “TCP/IP, DNS, HTTP” equivalents for autonomous agents ² ¹⁰. If widely adopted, it could mean that a planning agent from Vendor A can seamlessly delegate sub-tasks to agents from Vendors B and C, crossing organizational boundaries, all while behaving like a trusted, coherent system. In the words of one analysis: *“AGNTCY presents the most ambitious vision... aiming to build a comprehensive, secure, and scalable infrastructure (an ‘Internet of Agents’) through a suite of interconnected protocols and services like ACP and AGP.”* ⁵¹.

Sources: This summary is based on official AGNTCY documentation (docs.agntcy.org) and related technical articles [1](#) [14](#) [17](#) [2](#) [30](#) [38](#) , which describe the AGNTCY vision, components, and protocols in depth.

[1](#) [3](#) [4](#) [5](#) [6](#) [9](#) [13](#) [16](#) [31](#) [34](#) [39](#) [40](#) **Agntcy**

<https://docs.agntcy.org/>

[2](#) [11](#) [42](#) [43](#) [44](#) [46](#) [47](#) [49](#) [50](#) **Open Standards for AI Agents: A Technical Comparison of A2A, MCP, LangChain Agent Protocol, and AGNTCY**

<https://www.linkedin.com/pulse/open-standards-ai-agents-technical-comparison-a2a-mcp-jin-v7qve>

[7](#) [8](#) [10](#) [12](#) [30](#) [32](#) [33](#) [41](#) [45](#) [48](#) [51](#) **Comparing MCP, A2A, and AGNTCY in the AI Agent Ecosystem | by Omar Santos | Medium**

<https://becomingahacker.org/comparing-mcp-a2a-and-agntcy-in-the-ai-agent-ecosystem-f3234b85c475?gi=d59bc8b2fcf1>

[14](#) [18](#) [20](#) [21](#) [28](#) **Overview - Agntcy**

<https://docs.agntcy.org/dir/overview/>

[15](#) [17](#) [19](#) [22](#) [23](#) [24](#) [29](#) [37](#) [38](#) **Evolution of AI Agent Registry Solutions: Centralized, Enterprise, and Distributed Approaches**

<https://arxiv.org/html/2508.03095v3>

[25](#) [26](#) [27](#) **Features and Usage Scenarios - Agntcy**

<https://docs.agntcy.org/dir/scenarios/>

[35](#) [36](#) **Trust Model - Agntcy**

<https://docs.agntcy.org/dir/trust-model/>